

那么, 哪一个比较好呢? 是 member 函数 `clearEverything` 还是 non-member 函数 `clearBrowser`?

面向对象守则要求, 数据以及操作数据的那些函数应该被捆绑在一块, 这意味它建议 member 函数是较好的选择。不幸的是这个建议不正确。这是基于对面向对象真实意义的一个误解。面向对象守则要求数据应该尽可能被封装, 然而与直观相反地, member 函数 `clearEverything` 带来的封装性比 non-member 函数 `clearBrowser` 低。此外, 提供 non-member 函数可允许对 `WebBrowser` 相关机能有较强的包裹弹性 (packaging flexibility), 而那最终导致较低的编译相依度, 增加 `WebBrowser` 的可延伸性。因此在许多方面 non-member 做法比 member 做法好。重要的是, 我们必须了解其原因。

让我们从封装开始讨论。如果某些东西被封装, 它就不再可见。愈多东西被封装, 愈少人可以看到它。而愈少人看到它, 我们就有愈大的弹性去变化它, 因为我们的改变仅仅直接影响看到改变的那些人事物。因此, 愈多东西被封装, 我们改变那些东西的能力也就愈大。这就是我们首先推崇封装的原因: 它使我们能够改变事物而只影响有限客户。

现在考虑对象内的数据。愈少代码可以看到数据 (也就是访问它), 愈多的数据可被封装, 而我们也就愈能自由地改变对象数据, 例如改变成员变量的数量、类型等等。如何量测 “有多少代码可以看到某一块数据” 呢? 我们计算能够访问该数据的函数数量, 作为一种粗糙的量测。愈多函数可访问它, 数据的封装性就愈低。

条款 22 曾说过, 成员变量应该是 `private`, 因为如果它们不是, 就有无量数的函数可以访问它们, 它们也就毫无封装性。能够访问 `private` 成员变量的函数只有 class 的 member 函数加上 friend 函数而已。如果要你在一个 member 函数 (它不只可以访问 class 内的 `private` 数据, 也可以取用 `private` 函数、enums、typedefs 等等) 和一个 non-member, non-friend 函数 (它无法访问上述任何东西) 之间做抉择, 而且两者提供相同机能, 那么, 导致较大封装性的是 non-member non-friend 函数, 因为它并不增加 “能够访问

class 内之 private 成分”的函数数量。这就解释了为什么 clearBrowser(一个 non-member non-friend 函数)比 clearEverything(一个 member 函数)更受欢迎的原因:它导致 WebBrowser class 有较大的封装性。

在这一点上有两件事情值得注意。第一,这个论述只适用于 non-member non-friend 函数。friends 函数对 class private 成员的访问权力和 member 函数相同,因此两者对封装的冲击力道也相同。从封装的角度看,这里的选择关键并不在 member 和 non-member 函数之间,而是在 member 和 non-member non-friend 函数之间。(当然,封装并非唯一考虑。条款 24 解释当我们考虑隐式类型转换,应该在 member 和 non-member 函数之间抉择。)

第二件值得注意的事情是,只因在意封装性而让函数“成为 class 的 non-member”,并不意味着它“不可以是另一个 class 的 member”。这对那些习惯于“所有函数都必须定义于 class 内”的语言(如 Eiffel, Java, C#)的程序员而言,可能是个温暖的慰藉。例如我们可以令 clearBrowser 成为某工具类(utility class)的一个 static member 函数。只要它不是 WebBrowser 的一部分(或成为其 friend),就不会影响 WebBrowser 的 private 成员封装性。

在 C++, 比较自然的做法是让 clearBrowser 成为一个 non-member 函数并且位于 WebBrowser 所在的同一个 namespace(命名空间)内:

```
namespace WebBrowserStuff {  
    class WebBrowser { ... };  
    void clearBrowser(WebBrowser& wb);  
    ...  
}
```

然而这不只是为了看起来自然而已。要知道,namespace 和 classes 不同,前者可跨越多个源码文件而后者不能。这很重要,因为像 clearBrowser 这样的函数是个“提供便利的函数”,如果它既不是 members 也不是 friends,就没有对 WebBrowser 的特殊访问权力,也就不能提供“WebBrowser 客户无法以其他方式取得”的机能。举个例子,如果 clearBrowser 不存在,客户端就只好自行调用 clearCache, clearHistory 和 removeCookies。

一个像 WebBrowser 这样的 class 可能拥有大量便利函数,某些与书签(bookmarks)有关,某些与打印有关,还有一些与 cookie 的管理有关……通常大多数客户只对其中某些感兴趣。没道理一个只对书签相关便利函数感兴趣的客户却与……呃……例如一

个 cookie 相关便利函数发生编译相依关系。分离它们的最直接做法就是将书签相关便利函数声明于一个头文件, 将 cookie 相关便利函数声明于另一个头文件, 再将打印相关便利函数声明于第三个头文件, 依此类推:

```
//头文件 "webbrowser.h" — 这个头文件针对 class WebBrowser 自身
// 及 WebBrowser 核心机能。
namespace WebBrowserStuff {
class WebBrowser { ... };
    ...           //核心机能, 例如几乎所有客户都需要的
                // non-member 函数。
}

//头文件 "webbrowserbookmarks.h"
namespace WebBrowserStuff {
    ...           //与书签相关的便利函数
}

//头文件 "webbrowsercookies.h"
namespace WebBrowserStuff {
    ...           //与 cookie 相关的便利函数
}
...
```

注意, 这正是 C++ 标准程序库的组织方式。标准程序库并不是拥有单一、整体、庞大的 `<C++StandardLibrary>` 头文件并在其中内含 `std` 命名空间内的每一样东西, 而是有数十个头文件 (`<vector>`, `<algorithm>`, `<memory>` 等等), 每个头文件声明 `std` 的某些机能。如果客户只想使用 `vector` 相关机能, 他不需要 `#include <memory>`; 如果客户不想使用 `list`, 也不需要 `#include <list>`。这允许客户只对他们所用的那一小部分系统形成编译相依 (见条款 31, 其中讨论降低编译依存性的其他做法)。以此种方式切割机能并不适用于 `class` 成员函数, 因为一个 `class` 必须整体定义, 不能被分割为片片片段。

将所有便利函数放在多个头文件内但隶属同一个命名空间, 意味客户可以轻松扩展这一组便利函数。他们需要做的就是添加更多 `non-member non-friend` 函数到此命名空间内。举个例子, 如果某个 `WebBrowser` 客户决定写些与影像下载相关的便利函数, 他只需要在 `WebBrowserStuff` 命名空间内建立一个头文件, 内含那些函数的声明即可。新函数就像其他旧有的便利函数那样可用且整合为一体。这是 `class` 无法提供的另一

个性质，因为 `class` 定义式对客户而言是不能扩展的。当然啦，客户可以派生出新的 `classes`，但 `derived classes` 无法访问 `base class` 中被封装的（即 `private`）成员，于是如此的“扩展机能”拥有的只是次级身份。此外一如条款 7 所说，并非所有 `classes` 都被设计用来作为 `base classes`。

请记住

- 宁可拿 `non-member non-friend` 函数替换 `member` 函数。这样做可以增加封装性、包裹弹性（`packaging flexibility`）和机能扩充性。

条款 24：若所有参数皆需类型转换，请为此采用 `non-member` 函数

Declare `non-member` functions when type conversions should apply to all parameters.

我在导读中提过，令 `classes` 支持隐式类型转换通常是个糟糕的主意。当然这条规则有其例外，最常见的例外是在建立数值类型时。假设你设计一个 `class` 用来表现有理数，允许整数“隐式转换”为有理数似乎颇为合理。的确，它并不比 C++ 内置从 `int` 至 `double` 的转换来得不合理，而还比 C++ 内置从 `double` 至 `int` 的转换来得合理些。假设你这样开始你的 `Rational class`：

```
class Rational {
public:
    Rational(int numerator = 0,           //构造函数刻意不为 explicit;
             int denominator = 1);       //允许 int-to-Rational 隐式转换。
    int numerator() const;               //分子 (numerator) 和分母 (denominator)
    int denominator() const;            //的访问函数 (accessors) —见条款 22。
private:
    ...
};
```

你想支持算术运算诸如加法、乘法等等，但你不确定是否该由 `member` 函数、`non-member` 函数，或可能的话由 `non-member friend` 函数来实现它们。你的直觉告诉你，当你犹豫就该保持面向对象精神。你知道有理数相乘和 `Rational class` 有关，因此很自然地似乎该在 `Rational class` 内为有理数实现 `operator*`。条款 23 曾经反直觉地主张，将函数放进相关 `class` 内有时会与面向对象守则发生矛盾，但让我们先把那放在一

旁, 先研究一下将 `operator*` 写成 `Rational` 成员函数的写法:

```
class Rational {
public:
    ...
    const Rational operator* (const Rational& rhs) const;
};
```

(如果你不确定为什么这个函数被声明为此种形式, 也就是为什么它返回一个 `const by-value` 结果但接受一个 `reference-to-const` 实参, 请参考条款 3, 20 和 21。)

这个设计使你能够将两个有理数以最轻松自在的方式相乘:

```
Rational oneEighth(1, 8);
Rational oneHalf(1, 2);
Rational result = oneHalf * oneEighth;    //很好
result = result * oneEighth;              //很好
```

但你还不满足。你希望支持混合式运算, 也就是拿 `Rationals` 和……嗯……例如 `ints` 相乘。毕竟很少有什么东西会比两个数值相乘更自然的了——即使是两个不同类型的数值。

然而当你尝试混合式算术, 你发现只有一半行得通:

```
result = oneHalf * 2;          //很好
result = 2 * oneHalf;          //错误!
```

这不是好兆头。乘法应该满足交换律, 不是吗?

当你以对应的函数形式重写上述两个式子, 问题所在便一目了然了:

```
result = oneHalf.operator*(2);    //很好
result = 2.operator*(oneHalf);    //错误!
```

是的, `oneHalf` 是一个内含 `operator*` 函数的 `class` 的对象, 所以编译器调用该函数。然而整数 2 并没有相应的 `class`, 也就没有 `operator*` 成员函数。编译器也会尝试寻找可被以下这般调用的 non-member `operator*` (也就是在命名空间内或在 `global` 作用域内):

```
result = operator*(2, oneHalf);    //错误!
```

但本例并不存在这样一个接受 `int` 和 `Rational` 作为参数的 non-member `operator*`, 因此查找失败。